ASSIGNMENT 1

Comparitive study of AI Agent Frameworks

(Langchain | AutoGen | CrewAi | Agno | LlamaIndex | n8n | OpenClaw)

Name :- Vaishnavi Vijay Punde

Roll_No :- BSC23DS121(B)

Subject :- Natural Langugae Processing:- Natural Langugae Processing

INTRODUCTION

AI agent frameworks are software tools that help developers build intelligent systems using Large Language Models (LLMs). These systems, called AI agents, can do more than just answer questions — they can break down tasks into steps, use external tools, store memory, and in some cases, coordinate with other agents to complete a goal.

As AI adoption has grown, many frameworks have been developed to support different types of agent workflows. Each framework has a different design approach — some focus on pipeline-based execution, some on multi-agent collaboration, some on connecting LLMs to data, and some on visual automation without writing code.

This report compares seven frameworks: LangChain, AutoGen, CrewAI, Agno, LlamaIndex, n8n, and OpenClaw. The goal is to study each framework individually and then compare them based on common criteria such as architecture, ease of use, multi-agent support, memory handling, RAG capability, LLM provider support, and suitable use cases.

METHODOLOGY

To compare these frameworks, I followed a structured approach:

Step 1 — Individual Study

I studied each framework one by one before comparing them. I read the official documentation for LangChain, AutoGen, CrewAI, and LlamaIndex. For Agno and OpenClaw, I referred to their GitHub repositories and community write-ups since their documentation is still developing.

Step 2 — Selecting Comparison Criteria

I identified the following criteria for comparison: architecture type, ease of use, multi-agent support, memory types, RAG support, LLM provider compatibility, production readiness, learning curve, customization, and ideal use cases.

Step 3 — Using AI Tools for Research

I used Claude AI and ChatGPT to understand complex concepts and get initial summaries. Perplexity AI was used to verify specific details. Google Search helped me find architecture diagrams and official documentation. After reading AI responses, I rewrote the content in my own words and verified key points against documentation before including them.

Step 4 — Building Comparison Tables

After collecting information for all seven frameworks across all criteria, I organized the data into comparison tables. Ratings in the use case matrix were assigned based on how well each framework's architecture fits a given task type.

FRAMEWORK OVERVIEWS

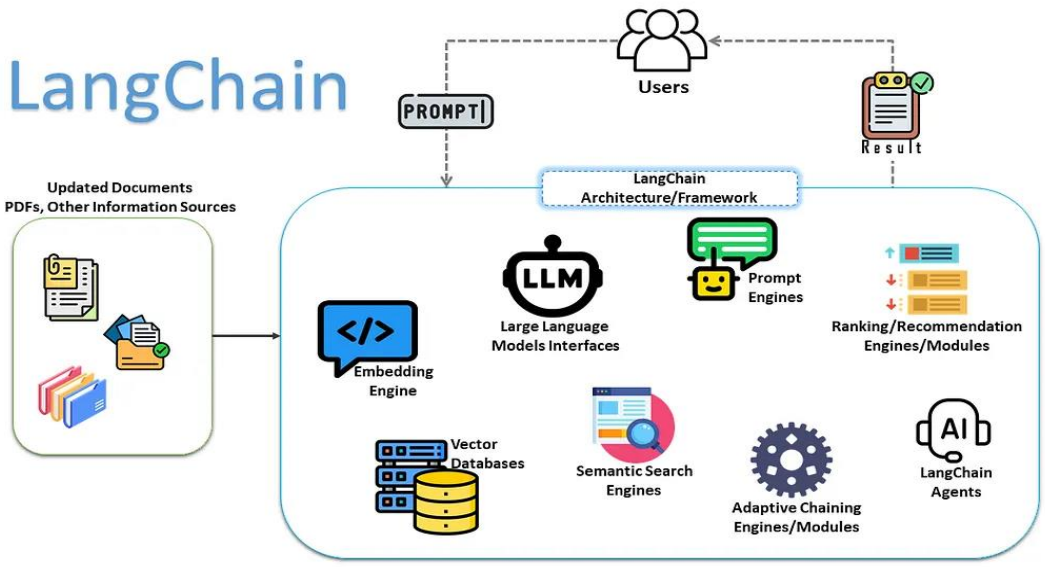
1.Langchain

GitHub Stars	~90K+
Language	Python, JavaScript
Released	2022
Company	LangChain Inc.

LangChain is one of the earliest frameworks for building LLM-powered applications. The main concept is the chain — a sequence of connected components where each component performs a specific role such as accepting a prompt, querying an LLM, calling a tool, or reading from memory. These components are linked together to form a complete workflow.

LangChain supports a wide range of integrations — vector stores, document loaders, output parsers, retrievers, and more. For proper agentic behavior (where an agent decides its own steps), LangChain relies on an extension called LangGraph, which adds graph-based execution on top of the chain model.

- Langchain Architecture



LangChain is best suited for developers who need deep integration with multiple external tools and APIs. The learning curve is high due to the number of components, but the ecosystem is the largest among all frameworks compared here.

- Strengths: Largest ecosystem, strong community, broad integrations
- Limitations: Complex to learn; full agent behavior needs LangGraph

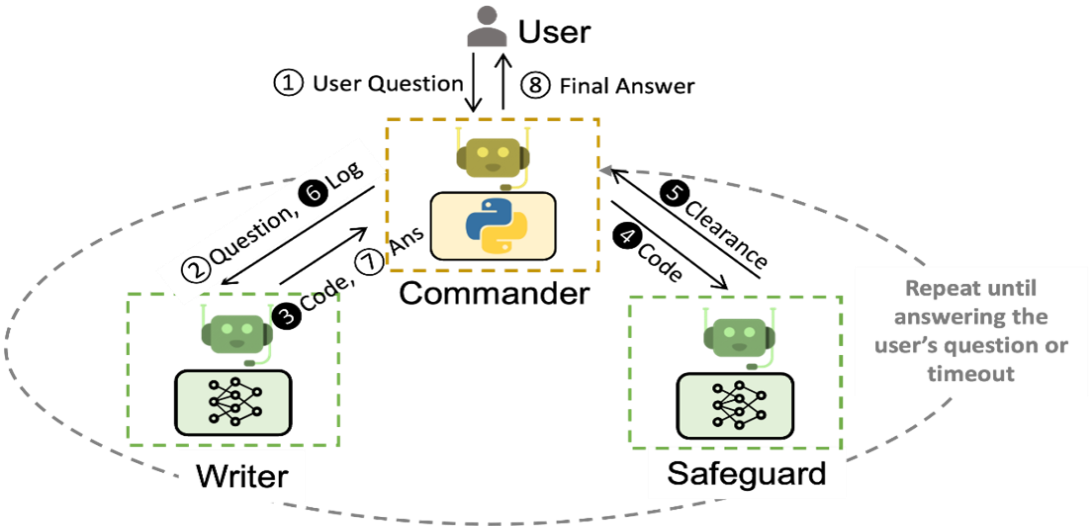
2.AutoGen

GitHub Stars	~40K+
Language	Python
Released	2023
Company	Microsoft Research

AutoGen is developed by Microsoft Research and is built around multi-agent conversation. Instead of a single agent working alone, AutoGen creates multiple agents that communicate with each other in a structured group chat to complete a task. Each agent has a defined role — for example, a UserProxy agent that initiates tasks and an AssistantAgent powered by an LLM.

AutoGen supports autonomous task execution, human-in-the-loop interaction, and code execution. Agents can write code, run it, check the output, and retry if needed — all within the same conversation loop. This makes AutoGen particularly strong for coding, research, and reasoning tasks.

- AutoGen Multi-agent Architecture


User
① User Question
⑧ Final Answer
② Question, ⑥ Log
③ Code, ⑦ Ans
④ Code
⑤ Clearance
Commander
Writer
Safeguard
Repeat until answering the user's question or timeout

- Strengths: Strong multi-agent support, excellent for code generation and research tasks
- Limitations: More complex to configure than CrewAI; conversation loops can sometimes go off track

3.CrewAI

GitHub Stars	~25K+
Language	Python
Released	2024
Company	CrewAI Inc.

CrewAI is a role-based multi-agent framework where agents are organized like members of a team. Each agent is assigned a specific role (such as Researcher, Writer, or Reviewer), a goal, and a set of tasks. The framework then coordinates these agents to work together toward a shared objective.

Compared to AutoGen, CrewAI is more structured and predictable because each agent's responsibility is clearly defined upfront. This also makes it easier for beginners to understand and implement. CrewAI is well-suited for content generation pipelines, multi-step research tasks, and any workflow that can be divided into clear roles.

- Strengths: Easy to learn, clean role-based structure, good for task delegation
- Limitations: Less flexible than LangChain for custom integrations; not ideal for low-level coding tasks

4. Agno

GitHub Stars	~15K+
Language	Python
Released	2024
Company	Agno AI

Agno (previously known as Phidata) is a lightweight, model-agnostic agent framework. It is designed to keep things simple — a working agent can be set up in just a few lines of code. It supports multiple LLM providers and is not tied to any specific one, which gives it good flexibility.

Agno supports multi-agent workflows, structured outputs, and tool calling. Because of its minimal setup and fast execution, it has become popular for prototyping and building lightweight AI applications. It gained notable traction in 2025 among developers who found larger frameworks like LangChain too heavy for simple use cases.

- Strengths: Minimal setup, lightweight, fast, model-agnostic
- Limitations: Smaller community and ecosystem compared to LangChain; fewer built-in integrations

5. LlamaIndex

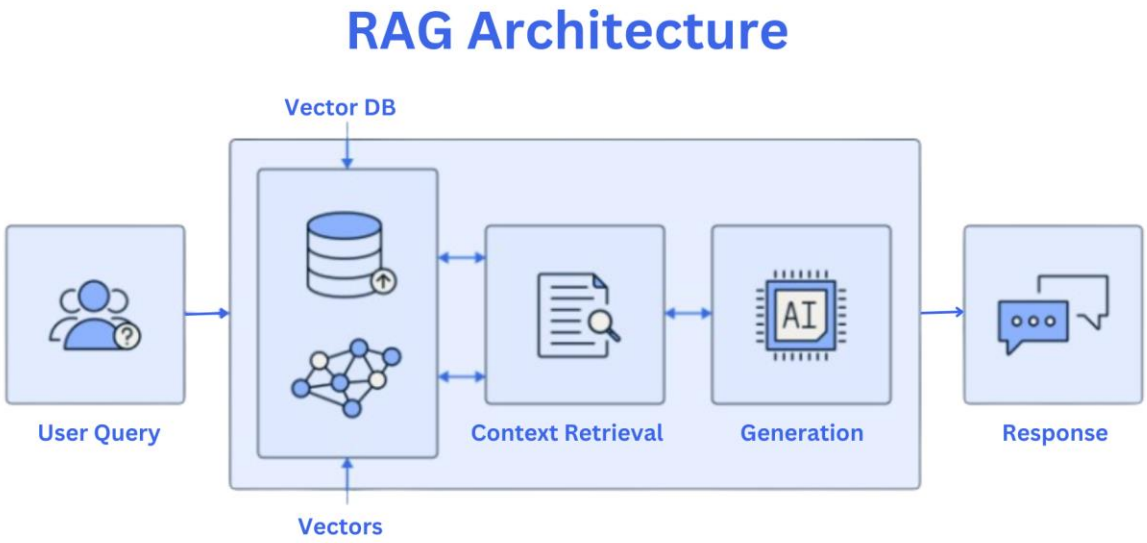
GitHub Stars	~37K+
Language	Python, TypeScript
Released	2022
Company	LlamaIndex Inc.

LlamaIndex started as GPT Index — a library built specifically to connect LLMs with external private data such as PDFs, databases, and APIs. Its core mechanism is Retrieval-Augmented

Generation (RAG): documents are loaded, split into chunks, converted into vector embeddings, and stored in an index. When a user submits a query, the most relevant chunks are retrieved and passed to the LLM as context before it generates a response.

Over time, LlamaIndex has expanded to support agents and query pipelines, but its primary strength remains in data-grounded applications. It is the most purpose-built framework for use cases that involve answering questions from private documents or structured data.

- LlamaIndex RAG Architecture



- Strengths: Best-in-class RAG, excellent for document QA and enterprise search
- Limitations: Less suited for pure multi-agent orchestration or coding tasks

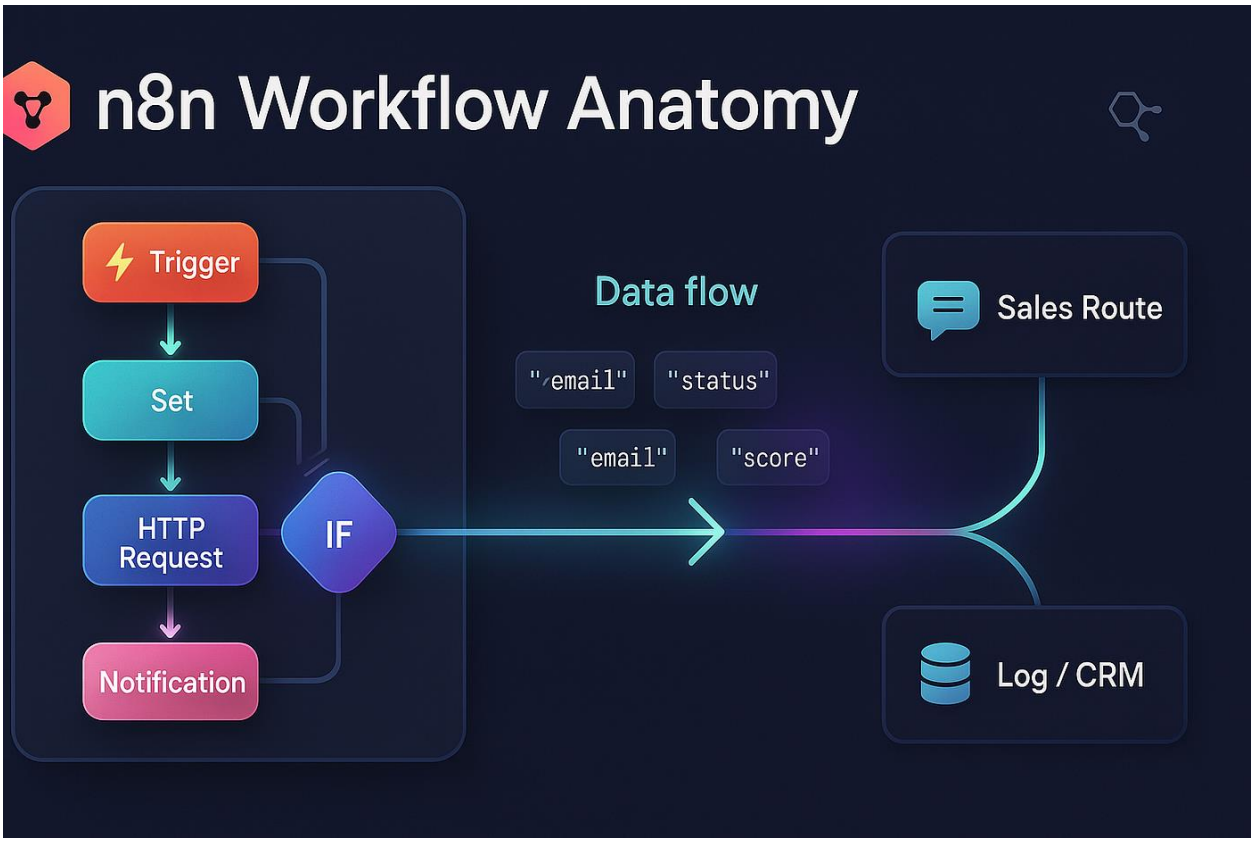
6. n8n

GitHub Stars	~50K+
Interface	Visual — Node-based (No-code / Low-code)
Released	2019
Company	n8n GmbH

n8n is different from all other frameworks in this list because it requires no programming. It is a visual, node-based workflow automation platform where users connect nodes on a canvas to build automated pipelines. Each node represents a step — an API call, a conditional check, an LLM query, a database write, etc.

n8n added native AI node support, which allows users to include LLM calls, vector search, and agent behavior as part of larger automated workflows — all through the UI. This makes it accessible to non-developers and suitable for business automation use cases.

n8n Visual Workflow Architecture



- Strengths: No coding required, easy to use, excellent for business process automation
- Limitations: Limited deep AI reasoning; complex agentic behaviors are difficult to implement

7.OpenClaw

GitHub Stars	~5K+
Language	Python
Released	2024
Company	Open-source Community

OpenClaw is an emerging open-source framework focused on structured, deterministic agent pipelines. Unlike LangChain or AutoGen where agents make dynamic decisions at runtime, OpenClaw uses Directed Acyclic Graphs (DAGs) to define the execution flow. Each node in the graph is a clearly defined agent action — making the pipeline predictable and testable.

This approach is particularly useful for enterprise scenarios where auditability and reliability are important. Since every step is pre-defined, it is easy to trace exactly what happened and why. The tradeoff is that OpenClaw is less flexible when a workflow needs to adapt dynamically. Documentation is still limited as the framework is early in development.

- Strengths: Deterministic execution, auditable pipelines, good for enterprise workflows
- Limitations: Still maturing; small community; limited documentation

COMPARISON TABLES

1.Core Feature Comparison

Framework	Architecture	Language	Ease of Use	Multi-Agent	Best Use Case	Prod. Ready
LangChain	Chain-based	Python / JS	Medium	Limited*	Integrations & Pipelines	High
AutoGen	Conversation-based	Python	Medium	Native	Research & Code Gen	High
CrewAI	Role-based	Python	Easy	Native	Team Workflows	Medium-High
Agno	Lightweight Agent	Python	Easy	Supported	Rapid Prototyping	Medium
LlamaIndex	RAG-based	Python / TS	Medium	Partial	Document & Data AI	High
n8n	Node / Workflow	No-code UI	Very Easy	No	Business Automation	High
OpenClaw	DAG Pipeline	Python	Medium	Strong	Enterprise Pipelines	Growing

LangChain multi-agent support requires the LangGraph extension. Production readiness is assessed based on adoption in real-world deployed systems as of 2024-2025.

2.LLM Provider Support

LLM Provider	LangChain	AutoGen	CrewAI	Agno	LlamaIndex	n8n	OpenClaw
OpenAI GPT	✓	✓	✓	✓	✓	✓	✓
Anthropic Claude	✓	✓	✓	✓	✓	✓	Partial
Google Gemini	✓	✓	✓	✓	✓	✓	Partial
Local (Ollama)	✓	✓	✓	✓	✓	Partial	✓
Mistral / Groq	✓	✓	✓	✓	✓	✓	Partial
Azure OpenAI	✓	✓	Partial	✓	✓	✓	✓

✓ = Full native support | Partial = Works with limitations or workarounds. All major frameworks support primary commercial providers. Key differentiator is local model support (e.g., Ollama) for privacy-sensitive deployments.

3. Memory,RAG and Technical Details

Feature	LangChain	AutoGen	CrewAI	Agno	LlamaIndex	n8n	OpenClaw
Memory Types	Buffer/Summary/Vector	Conversation	Short + Long	In-memory/DB	Chat + Index	Workflow State	Session-based
RAG Support	Good	Basic	Good	Good	Best-in-class	Via Nodes	Moderate
Learning Curve	High	Medium	Low–Medium	Low	Medium	Very Low	Medium
Customization	High	High	Medium	Medium	Medium	Low	High
GitHub Stars	~90K+	~40K+	~25K+	~15K+	~37K+	~50K+	~5K+

ANALYSIS AND KEY OBSERVATIONS

No Single Best Framework

The comparison shows clearly that no single framework is best for all situations. Each one is designed for a specific type of problem. The right choice depends on the use case, the development team's skill level, and system requirements.

RAG vs. Agent Orchestration

There is a clear divide between frameworks focused on RAG (connecting LLMs to data) and those focused on agent orchestration (coordinating multiple agents). LlamaIndex leads the RAG category. AutoGen and CrewAI lead in orchestration. LangChain and Agno try to cover both, but with different levels of depth

Code-First vs. No-Code

n8n is the only no-code option in this list. Its inclusion highlights that AI automation is not limited to developers. For business teams that need AI-assisted automation without writing code, n8n is a strong choice.

Lightweight vs. Full-Featured

Agno represents a trend toward lightweight frameworks. As LangChain has grown in complexity, many developers prefer a simpler alternative for small to mid-scale projects. Agno addresses this gap well.

Deterministic vs. Dynamic Execution

Most frameworks allow agents to make dynamic decisions at runtime. OpenClaw takes the opposite approach — fully structured DAG-based execution. This makes it more suitable for enterprise environments where every step must be traceable and auditable.

ELABORATION

How Did I Compare All the Frameworks?

I compared all seven frameworks using a structured, criteria-based approach. First, I listed the evaluation criteria: architecture type, ease of use, multi-agent support, memory handling, RAG capability, LLM provider support, production readiness, learning curve, and suitable use cases.

For each framework, I collected information from official documentation, GitHub repositories, and community resources. I then filled in each criterion for each framework based on what I read. For the use case matrix, I assessed each framework against specific task types based on its architectural design — for example, LlamaIndex scores highest for document QA because it is built specifically for that, while AutoGen scores highest for coding agents because of its multi-agent conversation and code execution capability.

Once all data was collected, I organized it into comparison tables for a clear side-by-side view and drew observations from the patterns across the data.

Which AI Tools Did I Use?

- Claude AI (Anthropic) — Used as primary research assistant to understand framework concepts, architecture details, and generate initial summaries.
- ChatGPT (OpenAI GPT-4) — Used for cross-referencing information and getting alternative explanations for complex topics.
- Perplexity AI — Used for verifying specific facts such as GitHub star counts and release years, as it cites sources.
- Google Search — Used to find official documentation, architecture diagrams, and developer write-ups.

What Prompts Did I Give to AI?

I did not use single prompt. I used multiple prompts. then combined the information to create the final comparison. Compare AI agent frameworks like LangChain, AutoGen, CrewAI, LlamaIndex, and n8n based on architecture, use cases, scalability, and ease of use. Give diAerences in table format and explain real-world use cases. I am comparing AI agent frameworks for a college assignment. Explain LangChain, AutoGen, CrewAI, LlamaIndex, and n8n architecture and give diagram Create detailed master comparison tables for AI agent frameworks like LangChain, AutoGen, CrewAI, Agno, LlamaIndex, n8n, and OpenClaw.

Did I Read the Response or Directly Copy-Paste?

I carefully read the information Some responses I have copied and pasted, and some responses I have created and written myself. I did not directly copy-paste the AI responses. First, I read all the answers carefully and tried to understand the concepts. Some parts were diAicult checked other sources to understand better.

CONCLUSION

This assignment involved studying and comparing seven AI agent frameworks: LangChain, AutoGen, CrewAI, Agno, LlamaIndex, n8n, and OpenClaw. Each framework represents a different approach to building AI-powered systems.

LangChain is the most established and feature-rich framework, best for complex integrations. AutoGen and CrewAI are strong choices for multi-agent systems — AutoGen for technical and coding tasks, CrewAI for structured role-based workflows. Agno is suitable for lightweight, fast prototyping. LlamaIndex is the best option when the application involves querying private documents or structured data. n8n is ideal for business automation without writing code. OpenClaw is a newer framework targeting enterprise use cases that require auditable, deterministic execution.

The main learning from this comparison is that framework selection should be driven by the use case and the team's requirements rather than popularity alone. Each framework solves a specific type of problem, and understanding those differences is key to making the right choice.